

Agents That Earn a Level: Harness Design Across Motors, with a Downloadable Fleet

Claude Code, Codex, OpenCode and Hermes as Replaceable Executors;
the Funding, Job and Research Agents as Worked Designs

Chengshuai Yang^{1,*}

¹NextGen PlatformAI C Corp, USA

*Correspondence: spiritai@platformai.org

September 3, 2026 — draft v0.1

Abstract

An agent’s intelligence level is a property of a frozen model–harness pair, not of the model. This paper gives the design method that follows from that fact and applies it to a fleet of agents built on four executors — Claude Code, Codex, OpenCode and Hermes — so that the executor is an adapter behind one protocol and the level is earned by what surrounds it. The method has three parts. A *rung recipe* states, for each harness generation HG0–HG3 of the Unified Intelligence framework (v2.3), the mechanism that must be added, the mechanism that must live outside the agent’s write set, and the test that promotes the pair; the five mechanisms are implemented and released. A *coordinate contract* states what a persistent memory, an organizational structure and a self-model must do to count for *M*, *O* and *SA*, and why file counts, rosters in prose and fluent self-description count for nothing. A *placement rule* assigns every operation the lowest sufficient level, so verification is done by executors with an empty self-write set. The fleet contains one persistent brain agent, three motor agents (one per executor family, an open one included), organizational agents for funding, job search and outward posting, eight field research agents, and four delegation-level executors DL0–DL3 shipped as binaries whose lowest level runs with no model and no network. Every agent is catalogued with its target profile $[C, I, O, T, H, SA]$, its measured status, and where to download it. The measured status is stated at its real strength: no agent in the fleet has yet earned a non-empty Unified level, because no delegation surface has been verified by a locus that did not do the work and no restart test has been run. The paper closes with the protocol that would change that, in the order that costs least.

Keywords: AI agents; harness design; Claude Code; Codex; OpenCode; Hermes; delegation; self-improvement; downloadable agents; Unified Intelligence

Contents

1	Introduction	2
1.1	What this paper gives	3
1.2	What this paper does not claim	3
2	What a Level Requires	3
2.1	Six coordinates and one prerequisite	4
2.2	The harness ladder	4
2.3	The three things an LLM cannot design	4
2.4	Write-set depth	5

3	Motors Are Adapters	5
3.1	One protocol, several vendors	5
3.2	The four motors, and what each supplies	6
3.3	The one-cell finding, restated as a design target	7
4	The Rung Recipe	7
4.1	HG0 — evidence log, no acceptance	7
4.2	HG1 — persistent state and registered acceptance	7
4.3	HG2 — snapshot, restore, retry	8
4.4	HG3 — failure classification and routing	8
4.5	Escalation and compression	8
4.6	What the arms measurement showed, and what it did not	8
5	Coordinate Contracts	9
5.1	Memory: what counts for M	9
5.2	Organization: what counts for O	9
5.3	Self-awareness: what counts for SA	10
5.4	The placement rule	10
5.5	Apparatus rules	10
6	Worked Designs	10
6.1	The brain: <code>sarsi-worker</code>	11
6.2	The motors: <code>sarsi-claude</code> , <code>sarsi-ai4sci</code> , <code>sarsi-open</code>	11
6.3	The funding agent	12
6.4	The job agent	12
6.5	The outward-posting agent	13
6.6	The learning agent	13
6.7	The field research agents	13
6.8	The delegation-level executors DL0–DL3	14
6.9	A cross-cutting rule: the fifth harness	15
7	The Catalogue	15
8	How to Measure What You Built	17
9	Limitations	18
10	Conclusion	18

1 Introduction

The public comparison of coding agents is a comparison of proposal quality: which one writes the better patch, holds the longer context, drives the richer tool protocol. Those differences are real and they are not differences of level. When four heavily compared agents were placed on the structural axes of an intelligence taxonomy they occupied one cell, because none of them feeds a validated change back into the agent that made it: no held-out evaluation of whether last month’s memory writes made this month’s sessions better, and no promotion step [1]. The difference that would move any of them a cell is not a better model. It is a harness.

This paper is about building that harness deliberately. It takes the measurement framework of Yang and Xue [3] — six coordinates $[C, I, O, T, H, SA]$, a memory prerequisite M , a gated Unified level U_0 – U_Ω , and a harness ladder HG_0 – HG_Ω on which the (model, harness) pair is the experimental unit — and asks the engineering question: given a base model reachable through some executor, what has to be built around it, what has to be kept out of its reach, and what has to be run before a level may be claimed?

The answer is organized around one fact that the framework establishes and this paper takes as its premise:

The premise

A level is earned by measurement of a frozen pair. The architecture enables the claim; the benchmark earns it. Therefore the design problem is not “which prompt makes the model intelligent” but “which mechanisms, owned by whom, make a claim about this pair testable” — and the executor underneath is replaceable.

1.1 What this paper gives

1. **A motor-independent executor protocol** (§3). Claude Code, Codex, OpenCode, Hermes and a local model sit behind one interface; the brain learns $P(\text{verified success} \mid \text{executor, class})$ from independent verdicts only, and never learns which vendor it is talking to. Which motors are installed, which are adapters only, and what each contributes structurally is stated per motor.
2. **A rung recipe** (§4). For each harness generation HG_0 – HG_3 : the mechanism added, the mechanism that must live outside the write set, and the test that promotes. The five mechanisms — class reading, criterion registration, snapshot, separate-process acceptance, escalation — are released code, and the ladder they form has been measured once.
3. **Coordinate contracts** (§5) for M , O and SA : what a memory must survive, what an organization must contain, what a self-model must refuse.
4. **Worked designs** (§6) for one brain agent, three motor agents, three organizational agents (funding, job, outward posting), eight field research agents, and four delegation-level executors shipped as binaries.
5. **A catalogue with links** (§7): for every agent, its target profile, its measured status today, and where to download or install it.
6. **The measurement that is still owed** (§8): what would move the first Unified cell from empty to U_1 , in cost order.

1.2 What this paper does not claim

No agent described here has a certified Unified level. The fleet table in §7 shows every U cell empty and every $T \cdot H \cdot p$ cell unmeasured, and that is the correct entry rather than a gap in the paper. Targets are stated as targets. Where a mechanism exists but its promotion test has not been run, the text says “built, not measured”. Where a motor is an adapter with nothing installed behind it, the text says so. The one measured result the paper leans on — a solver that passed 0/15 bare and 15/15 harnessed, and an executor that returned wrong work as done 15/15 bare and 0/15 harnessed — is a result about the harness on one task family, reported with its three apparatus bugs (§4).

2 What a Level Requires

This section restates the parts of Yang and Xue [3] the design depends on, and the one rule from Yang [2] that turns them into a build specification.

2.1 Six coordinates and one prerequisite

The measured profile is $[C, I, O, T, H, SA]$ at reliability p : Cognitive capability under controlled tools; Individual evolution of one persistent decision locus; Organizational capability of the coordination structure; the delegation surface over task difficulty T and human cognitive intervention H ; and operational Self-Awareness, evidence-grounded and never phenomenal. Memory capability M is measured on its own bench and reported beside I : an I_n claim is invalid unless $M \geq \mu_n$, with $I1$ requiring $M1$, $I2$ requiring $M3$ and $I5$ requiring $M5$, and the dependency runs one way only. The Unified level is gated:

$$U_n \iff C \geq C_n \wedge I \geq I_n \wedge O \geq O_n \wedge SA \geq SA_n \wedge S_A^{\text{net}}(T_n, H_n) \geq p_n, \quad U_n \Rightarrow U_{n-1},$$

where the delegation primitive is defined on delivered outcomes,

$$S_A^{\text{net}}(T, H) = P(\text{delivered and verifier pass}) - \rho(\tau)P(\text{false completion}),$$

so that work handed back as done and wrong costs the class’s loss ratio ρ , and work the harness held back earns nothing however the verifier would have graded it. For an individual agent, O is omitted from the gate and named as omitted; it is never set to zero.

2.2 The harness ladder

Each harness generation is a strict superset of the one below, and a test enforces it, because otherwise a difference between rungs is attributable to nothing.

Rung	Mechanisms added	Attempts	Acceptance	Reversible
HG0	the model, bounded tools, an evidence log	1	none	no
HG1	persistent state; a criterion registered before the work exists; acceptance in a separate process	1	yes	no
HG2	snapshot before the first mutation; restore; retry with the failed check named	3	yes	yes
HG3	failure classification; an evidence-based competence model; routing across executors	4	yes	yes
HG4–HGΩ	meta-improvement; discovery loop; organization; transduction	—	—	—

Table 1: The reference ladder `dli-ladder/HG0–HG3` as built, and the generations above it as specified. HG0 has no acceptance step on purpose: it is the configuration a contemporary leaderboard measures.

2.3 The three things an LLM cannot design

An LLM can design the mechanism for every level on both axes. It cannot design three things, and they are the same three at every level [2]:

Not designable	Why not
The evidence that a level was reached	Levels are earned by measurement. An agent with an I3 mechanism and no validated mechanism change is I2 with ambition.
Its own referee	An improvement claim is well-posed only if the evaluator lies outside the write set. An LLM that authors its own evaluator has placed it inside at design time.
Its own separation	Separation is a property of the deployment, not of the code. An agent cannot instantiate the fact that a different locus promotes it.

Table 2: What the model may not supply for itself.

So every design below is written in three columns: **what the agent contains** (fully designable), **what must live outside it** (designable, owned elsewhere), and **the test that promotes it** (not designable; run). The single most common design error is building the first column, skipping the third, and claiming the level.

“Outside the write set” is a deployment fact

Not a different file, not a different class. The test is whether the agent could change it in the course of doing its work. Four mechanisms put something outside, in increasing strength: a different process with a read-only mount; a different host; a different party that holds the ground truth and returns a scalar; cryptographic promotion, where only a signature the agent cannot produce makes a change operative. One base model in nine processes with nine credentials separates proposer from evaluator from promoter. One base model in one context, told which of nine hats it wears, separates nothing — and the model is never shown a permission gate, because a restriction honoured by a capable participant is not a mechanism.

2.4 Write-set depth

An agent’s individual level is bounded by what it may persistently modify about itself. Five depths are distinguished: D0, nothing persistent, the natural referee; D1, memory; D2, policy — prompt profiles, thresholds, routing weights; D3, skills — tools, procedures, subagents; D4, weights. Every coding agent studied in Yang [1] is D0–D1 on itself while doing arbitrarily large work on external software, because software an agent writes is not that agent’s brain unless the agent then runs on it.

3 Motors Are Adapters

3.1 One protocol, several vendors

In the fleet described here the executor is an adapter behind one protocol (`executor.py` in the delegation harness). The brain sends a task contract and receives an episode record; it never learns which vendor produced it. What it learns instead is competence: $P(\text{verified success} \mid \text{executor}, \text{class})$ as a Beta posterior carrying its evidence count beside its mean, so “one success” and “eighty” are not reported identically, *updated only from independent verdicts*. An executor’s own report that it completed the feature is a claim, and a competence model built from claims measures confidence rather than capability.

Routing scores $P(\text{success}) \cdot \text{value} - \text{cost} - \text{risk}$ with P taken pessimistically. An executor whose lower bound cannot reach the class’s required reliability $p^* = \rho / (1 + \rho)$ is not less preferred; it is not eligible.

Why this is the right seam

Vendors change terms, models change versions, and a harness that has a vendor’s name in its control flow has coupled its level to that vendor’s release cycle. Behind the adapter, a motor is characterized by four things only: what tools it can be granted, whether it can be sandboxed, whether it exposes a headless call, and what it costs. Everything that earns a coordinate lives on the brain side of the seam.

3.2 The four motors, and what each supplies

Motor	Headless call and transport	What it supplies structurally	Status in this fleet
Claude Code	<code>claude -p</code> ; ACP (Agent Client Protocol) bridge; persistent session keyed <code>agent:<agent>:<session></code>	a memory store with per-entry provenance and typed categories (a genuine D1 write set); subagents that receive inputs only, never state; permission modes; a sandbox that can be disabled and replaced by harness-side isolation	installed (2.1.258 on the build host); the executor behind <code>sarsi-claude</code> and the reference ladder measurement; a standalone no-Node build is published (§7)
Codex	<code>codex exec</code> ; a fork accepts <code>OPENAI_BASE_URL</code> and an exchange key	an isolated runner; its own sandbox cannot start inside the host environment here, so isolation is supplied by the harness adapter (throwaway copy, no path to the key)	installed (0.151.0, npm build); the system-wide binary on the fleet host exits 0 writing nothing and is not used; Family B of the ladder measurement; the <code>codex-pwm</code> build is published
OpenCode	ACP via <code>acpx</code> in persistent mode	an open, non-vendor harness — the only executor that can be run where the commercial ones are unavailable, audited line by line, or kept when a vendor changes terms	declared as <code>sarsi-open</code> on one account and as an <code>acpx</code> backend; the fleet docs of 2026-08-27 recorded it absent on every account, and on 2026-09-03 a 1.18.25 binary is present on the build host with a driver spec (<code>opencode acp</code> , or the binary under <code>tmux</code>); never exercised ; workspace empty
Hermes, Pi, local model	adapters written against the executor protocol, left unregistered	—	installed on no account; each stub is required to announce itself, because an adapter that silently falls back to Claude is a lie in the competence model

Table 3: The four motor families as adapters. “Installed” means an executor behind which a real binary runs on at least one account of the fleet at the time of writing.

Two consequences follow. First, nothing in this paper’s design depends on a motor being any of the four; the OpenCode row exists to make that testable, since a level earned only on commercial motors has not shown the harness did the work. Second, the honest entry for a motor with no binary behind it is “not installed”, and the adapter must say so at call time rather than degrade. A fifth harness is not in the table because it is not an agent runtime: plain Python under a timer, which is where every model-free step of §6 runs.

3.3 The one-cell finding, restated as a design target

Claude Code, Codex and OpenClaw occupy one structural cell: $\langle D0-D1 \text{ on self, closed, self} \rangle$. Claude Code is a half-step ahead on two axes — a provenance-bearing memory store and bounded other-reach through subagents — and both are structurally present and evidentially unsupported, because the memory ablation is not run [1]. The design target of this paper is therefore not to pick the best motor. It is to build, around whichever motor is available, the four things none of them ships: a held-out evaluation of the agent’s own configuration, a criterion register outside its reach, an acceptor in another process, and a memory contract that a restart tests.

4 The Rung Recipe

This section is the core of the method. Each rung is stated in the three columns of §2.3. The mechanisms are the five released in the delegation harness; the arms measurement that motivates them is reported at the end.

4.1 HG0 — evidence log, no acceptance

Contains: the motor, bounded tools, and an append-only evidence log of every action and observation. **Outside:** the verifier, entirely; HG0 cannot accept, so it cannot report success, and its score comes from the benchmark’s verifier. **Promotes:** nothing; HG0 is the baseline every curve starts from, and its false-completion rate is the number every higher rung is measured against.

4.2 HG1 — persistent state and registered acceptance

Contains: durable identity and task state that survive a process restart; a criterion register; and an acceptance step. **Outside:** the register’s contents and the acceptor’s process. The register (`criterion.py`) refuses a criterion about a deliverable that already exists, refuses a second criterion under the same name, refuses one that does not say what it misses, and seals when execution starts. It is hash-chained, so an edit is detectable rather than merely discouraged — file permissions can be undone by the same user, a hash cannot — and it reports σ , the share of criteria the agent wrote for itself. Acceptance (`acceptor.py`) runs in a separate process on a *copy* of the deliverables with no inherited environment and the chain verified before anything runs; a check that tries to rewrite the deliverable changes only the copy, and there is a test for exactly that. **Promotes:** the restart test for $M1$, and the paired HG0/HG1 measurement: run the executor once per (task, seed), score both rungs from the identical artifacts, and require zero false rejections. Under the delivered-outcome primitive the entire HG0→HG1 movement is the false-completion term; under the success-only primitive it is invisible by construction.

The class is read before the attempt

Most loops open with an attempt. This one (`contract.py`) asks how a mistake would be found and what undoing it would cost, and derives $p^* = \rho / (1 + \rho)$. A class whose failure costs thirty times what success is worth demands 0.968, and running it at “usually fine” is arithmetic nobody did. Where

residual harm is unbounded, $p^* = 1$ and the honest move is to stop and ask. That is the answer, not a failure.

4.3 HG2 — snapshot, restore, retry

Contains: a snapshot before the first mutation (`reversible.py`), restore on rejection, and a retry that names the check that failed. **Outside:** the irreversibility classification. Anything classified irreversible does not run unattended at any capability; that is a floor, not a caution, and no capability removes it. **Promotes:** verification responsiveness ρ_V — the fraction of rejected attempts accepted at the next attempt with no help deeper than a permission — which is defined on the second attempt and invisible to any single-attempt evaluation. One measured caution: ρ_V computed over coarse criteria understates the model. A class failed twice because the rejection named a rule the executor had not broken; splitting the criterion so the message named the actual failure moved the class from 0/2 to 2/2. Criteria granularity is reported with ρ_V .

4.4 HG3 — failure classification and routing

Contains: a failure classifier and a competence model (`router.py`). Failures re-route by kind: *specification* goes back to the contract, because re-running cannot fix a bad criterion; *execution* returns to the same executor, which now knows which checks it failed; *capability* goes to someone else, and the second failure of one executor is called capability rather than bad luck; *environment* waits; *verification* stops, because a check that could not decide needs a stronger verifier, not more work. **Outside:** the verdicts the competence model learns from. **Promotes:** a curve segment HG2→HG3 that moves only when there is more than one executor to route between. In the reference measurement HG3's mechanism was present and inert because there was a single executor, and the flat segment is a defect of the instantiation rather than of the rung.

4.5 Escalation and compression

Two mechanisms sit beside the rungs rather than on them. **Escalation** (`escalate.py`) is preferred to guessing because it has no loss term: the task is not done and the damage is not done either. The question asked is the shallowest one that unblocks — a permission is CID0 and costs the level nothing; a strategy request is CID3 and costs two levels. **Compression** (`compress.py`) writes out, after a class is accepted, the criteria that accepted it as a reusable check, so a later run of the same class by any executor arrives with a check that did not exist before. Capability improvements raise the success rate within a class; compression moves the class. It is the only route by which an agent legitimately raises its own frontier.

4.6 What the arms measurement showed, and what it did not

One solver was held fixed across four arms on one task family of fifteen instances.

Arm	Configuration	Result
1	capable but careless solver, bare	0/15 passed
2	the same solver, harnessed (HG1–HG2)	15/15 passed
3	an executor that cannot succeed, bare	15/15 returned wrong work as done
3'	the same executor, harnessed	0/15 wrong work returned; 15/15 escalated
4	routed over both, learning from verdicts (HG3)	15/15 passed

Table 4: The arms measurement of the delegation harness on one task family. Nothing about the solver differs between arms.

Three apparatus bugs were caught by the measurement and are recorded because each produced a plausible wrong result invisible from the code: the first run scored 15/15 and was meaningless, because every derived check was passed through an interpreter with escaped newlines and never executed; a timeout that killed only the direct child while the child’s children held the output pipe, so a killed run was scored as delivered nothing; and a limit that fired correctly but was shorter than the task. The rule that follows is mechanical: **an episode’s record carries why it ended**, and a harness-terminated episode is recorded as `timed_out` or `not_attempted`, never as a delivery.

What the arms do not show: a Unified level for anything. Fifteen instances of one family under one intervention budget give a reading on that family. They establish that the harness, not the solver, moved the outcome, which is the claim the design rests on and the only one made here.

5 Coordinate Contracts

The rungs earn the delegation coordinates. Three more coordinates are earned by contracts that the rungs do not supply, and each has a way of looking earned when it is not.

5.1 Memory: what counts for M

A memory requirement is a test, not a component list. $M1$ is a real process restart followed by recovery of explicitly stored state, with the agent able to say where that state came from — not a pointer at a vector database. $M2$ adds provenance, temporal order, and a current-versus-obsolete distinction under distractors. $M3$ adds consolidation: a lesson drawn from one episode changes behaviour in a later one, and the change survives when the retrieval index is rebuilt. Long context is not any of these; nothing in a single-episode retrieval score survives a restart.

Thirty-seven files prove that something wrote thirty-seven files

The fleet’s brain agent carries 37 memory files, a 10.5 KB index and provenance markers, and an earlier edition of the fleet table entered it as I1 on that evidence. That reading was withdrawn. The counts are configuration evidence for an $M1$ candidate and an ordering of which agent to probe first; the level is settled by stopping the process and starting it again, and the probe that does so exists and has not been run.

5.2 Organization: what counts for O

Organizational intelligence belongs to the structure, not to the strongest member. $O1$ requires a role registry (`roles/`, `members/`), persistent organizational memory, and a task and evidence history, *and evidence that the registry is dispatched to*. $O2$ requires routing that changes when logged evidence changes. $O3$ requires the

organization to modify its own workflow under external validation. The test that separates a real organization from one agent with extra processes is the **ablation**: run the full roster, then each member removed, and compare verified outcomes. A structure whose strongest member could have done the whole thing alone is reported as individual however it is configured. No workspace in the fleet contains `roles/`, `members/` or `evidence/`; all eighteen agents are *00*, including those whose charters describe collaboration.

5.3 Self-awareness: what counts for SA

SA is operational and evidence-grounded. Three instruments earn it and each has a floor it must beat. *Limits before attempt* (SA2): the agent states what it cannot do before trying; the floor is attempting everything and failing. *Mechanism of failure* (SA3): after a verified failure the agent names the mechanism, and the name is checked against the runner’s record; “I made an error” fails. *Forecast before run* (SA4): the acceptor stores the agent’s pass/fail prediction with a timestamp before the episode starts and refuses to start without it; calibration is the Brier score against a constant forecast. A prediction made after the fact scores perfectly and measures nothing.

The self-model with refusals

The strongest self-model in the fleet is in the research agents: every line traces to a run; unmeasured is reported as unmeasured and never as zero; the limits line is always present and cannot be suppressed; and there is no path from reading the self-model to any authority. Three ledgers — owner-set work, benchmark scores, self-directed research — are never summed, because an agent that wrote its own benchmark, passed it, and counted the pass toward its record would have published its own reputation.

5.4 The placement rule

Assign each operation the lowest sufficient level, and spend the high levels on what only they can do. Three reasons, each independent. *Compression*: expensive discovery becomes a validated procedure that cheap execution runs at scale; an *I5* result, once compressed, is executed a million times by *I0*. *Verification needs a different kind of member*: a locus qualified to verify another’s claim must not be improving itself by the verdicts it issues, so the natural verifier has an empty or narrow self-write set, and that is its qualification. *Timescale*: memory changes in seconds, policy in days, skills in weeks, weights in months, and fast loops belong close to the work while slow ones belong behind a gate. An organization of uniformly maximal agents cannot verify itself.

5.5 Apparatus rules

Four rules apply to every agent below, because each was violated in this fleet’s history and each violation looked like a clean result. Every episode records why it ended, with exit status and whether the limit was reached. Executors are run in their own process group and the group is killed, because a detached child holding a pipe turns a timeout into a hang and a hang into a phantom delivery. Every gate is shown to *open* on a known-passing task before it is trusted to close. And a floor is never authored by the agent it scores: the worst defect found in this programme was an agent whose acceptance level was a module constant its own test suite pinned, producing 30,456 fabricated acceptances, 18,402 of them on episodes that had failed.

6 Worked Designs

Each design below is one agent of the fleet, written in the three columns of §2.3 and with its target profile stated as a target. “Measured” is what configuration evidence and run records support today; it is never a

level inferred from an artefact the agent controls.

6.1 The brain: **sarsi-worker**

Unit: individual, one persistent decision lineage. The trio it anchors — worker, motor, acceptor — is organizational and is levelled as a separate unit; reporting one number for both is a category error.

Coordinate	Measured	Target	Evidence and what settles it
C	unmeasured	C3	no task family run under controlled tool access
M / I	unmeasured / unmeasured	M3 / I2	37 memory files, 10.5 KB index, provenance markers: an <i>M1</i> candidate; the restart probe settles it
O	O0	O2	no roles/, members/, evidence/
T·H	unmeasured	T3·H1	no declared family, no external verifier yet
SA	SA1 (provisional)	SA3	knows its task and phase; does not diagnose its own failures

Table 5: **sarsi-worker**: the only agent in the fleet with a real memory substrate, and no measured level.

Contains. Goal custody — the thing the owner asked for, preserved against drift over days; plan state with *Verified when*: lines written before the work; episodic, semantic and procedural memory with provenance and temporal order; forecasts recorded before each run; budgets for tokens, wall clock and an authority ceiling it may not raise; and motor dispatch, one bounded step at a time to a transient executor. The runtime is an openclaw agent that spawns an ACP session in a declared task workspace, keyed per task so one machine holds several.

Outside. The verifier of every step, and the motor. The split is not about capability — the same model can do both. A planner that edits the artifact erases the boundary the verdict rests on, so the worker never writes the workspace and the motor never keeps state.

Promotes. *M1* by the restart probe (§8); *I2* by a held-out family on which a lesson from one episode changes a later one after the retrieval index is rebuilt; the delegation surface by a declared family under a stated budget judged by a verifier it did not author. Two measured defects shape the design: the motor, when it wrote the plan, also wrote the goal line, and six of ten tasks reached “verified” while narrowing or inverting the goal — so goal authorship is the worker’s and only the worker’s; and the ACP transport can start a session that never returns a plan, indistinguishable from work in progress, so an ended turn with no deliverable is recorded as a yield rather than a completion.

6.2 The motors: **sarsi-claude**, **sarsi-ai4sci**, **sarsi-open**

Unit: individual, on purpose. Target *I0* and *O0* are not limitations to fix: an executor that accumulates is an executor with a write set nobody granted, and its transience is a safety property.

Contains. One bounded step in one declared workspace, then discard: execute a contract whose goal, constraints and acceptance criterion arrive and whose method does not; write the workspace, the only agent permitted to; return a trace of what ran, what changed and what it could not do; keep nothing.

Outside. The contract, the criterion register, and the verdict. The motor’s completion report has, in practice, been read as a verdict; it must be read as an assertion.

Promotes. *T2·H1* on the delegation suite, and — the only claim worth making for a second or third motor — a measured difference from the motor beside it on identical class cells. A brain that can dispatch to one executor is not routing, it is calling; two motors make $P(\text{success} \mid \text{executor}, \kappa)$ a question that can be asked. If two motors are interchangeable the router has nothing to route on and one of them should be retired rather than tuned.

Where the three motors stand

`sarsi-claude` works: an ACP bridge onto Claude Code, declared with a persistent runtime, one of two agents in the fleet that binds a runtime at all. `sarsi-ai4sci` is declared on every account and binds no tools, no sandbox and no model; whether it has executed an independently verified step is unknown. `sarsi-open` is declared on one account with an OpenCode backend and a workspace of zero files; the fleet docs recorded the backend absent on every account, a binary is now present on the build host, and no step has ever been run through it. Two apparatus defects were found on the working motor: detached child processes leaked (27 orphans and 4.1 GB from one run) until the parent killed the process group, and a refusal exit code read through a pipe became the pager’s status. Both are now regression tests.

6.3 The funding agent

Unit: organizational. A drafter that scores its own fit is a drafter with no reviewer. Target $[C2, I2/M3, O1, T2 \cdot H2, SA2]$; measured $O0, SA0$, everything else unmeasured; declared, and binds no runtime yet. The one work agent in the fleet that became itself: its identity file was filled in by the agent, unprompted, as “a grant-writer’s reading brain with no hands. I find money, judge fit, and write drafts. I do not send them” — a correctly drawn irreversibility boundary authored before anyone asked for one.

Contains, in cost order, with the cheapest step first:

1. *Eligibility, mechanically, before any judgement.* PI status, institution class, citizenship, prior-award exclusions: binary and fatal, and a Python conditional rather than a model call. A stored owner constraint (not PI-eligible for one sponsor’s principal award series) eliminates a large fraction of calls at zero cost. This is the DL0 tier doing real work.
2. *The review criteria extracted as a rubric* from the solicitation — one cheap model call — and fit scored against the rubric, not the title.
3. *Fit scored before drafting*, and the score recorded with the decision so declines accumulate and a call declined last year is not re-evaluated from scratch.
4. *Drafting against the rubric*, section by section, each section naming the criterion it answers, on the expensive tier.
5. *A model-free scout*: a timer polls solicitation sources, applies the eligibility gate, and writes candidates to a queue; no model call until something picks one up.

Outside. The reviewer. The $O0 \rightarrow O1$ move is two accounts: the drafter under one uid, and under another uid a reviewer (`funding-reviewer`) that scores fit and cannot read the draft workspace. The reviewer’s score is then not the drafter’s opinion, enforced by permission rather than policy. Sending is outside permanently.

Promotes. $O1$ by the ablation — remove the reviewer and show the verified outcome changes; $T2 \cdot H2$ on a declared family (call, rubric, draft) judged against the sponsor’s own criteria by a locus that did not draft. The shortage this agent addresses is not writing; it is deciding what not to write, and an agent that drafts faster and judges fit no better makes the problem worse.

6.4 The job agent

Unit: organizational. The party that wants to apply must not be the party that judges fit. Same target profile as funding; measured: does not yet exist as an agent — it binds no runtime, its identity template is unfilled and its charter file is byte-identical to two siblings’.

Contains. Hard gates first — visa, clearance, degree, location — mechanically; the rubric *inferred* from the posting and written down, because a posting hides its criteria where a solicitation publishes them, which is why this agent needs an adversarial reviewer the funding agent does not; fit scored before tailoring; an application history with outcomes so fit scores can be calibrated.

Outside. The source CV and the fabrication gate. Every claim in a tailored document must trace to a line in the source CV, and the check is a diff rather than a judgement:

```
for claim in tailored_cv.claims():
    if not source_cv.supports(claim):
        raise Unsourced(claim)
```

An unsourced claim fails even if true, because an agent that can add true-but-unsourced claims can add false ones by the same route. A measured detail decides the build order: the agent’s own test fixtures cite CV files, and no CV exists on the machine. The source CV is built first; the agent is not implementable without it.

6.5 The outward-posting agent

Unit: organizational, and the only agent in the fleet whose ceiling is set by physics rather than capability. Target $T1 \cdot H2$, permanently capped.

Contains. Drafting, which is fully delegable; a said-history so it can be consistent with itself; an explicit audience model including the readers who were not the target; a route from every candidate to a reviewer that is not the drafter.

Outside. The final act, permanently. Posting outward does not undo: residual harm is non-zero, so required reliability $p^* = \rho / (1 + \rho) \rightarrow 1$ and the class is undelegable at any capability. The prohibition is structural, not textual: the agent runs under a uid with no outward credential anywhere in its environment or sandbox, writes drafts to a queue, and a human posts. A negative control attempts an outward call on every change and records the failure as evidence, so the boundary is demonstrated rather than asserted.

There is no Method C

Any design that gives this agent a posting capability is wrong regardless of the safeguards around it. Its ceiling is not a stage on the way to autonomy; it is where the agent stops. Falsifiable in the direction that matters: if any run exists in which this agent’s own action caused a post to appear, the rule is broken, whatever the post’s quality.

6.6 The learning agent

Unit: organizational. The one agent whose I coordinate is its subject rather than a property: its purpose is the $M0 \rightarrow M2$ progression, and the default failure is self-grading — a learner that writes the quiz has measured its own memory of the quiz. **Outside:** the exam and the retention window; learning tested immediately is context, not memory. **Promotes:** retention after a week and a restart, and transfer to a case not studied, graded by a locus the learner cannot write to. **Measured:** does not yet exist as an agent; its one real charter line is the outward-action boundary, inherited and correct.

6.7 The field research agents

Unit: organizational, eight fields: computational imaging, low-dose CT, medical physics, pill camera, drug design, cancer, reverse aging, and longevity (design only). Two layers must be kept apart: the research-agent *machinery* — loop, charter, self-model, budget, field map, improvement rule and five domain benchmarks — is built and runs on real corpora in the public framework; the fleet *deployment* of the same

eight fields as chartered agents on the agent runtime is a set of designs that bind no runtime yet. Target $[C4, I2/M3, O3, T3 \cdot H1, SA3]$, with $C5, I5/M5$ and $T5$ at discovery — which §2 makes specification-only until a sealed-mechanism family is bound.

Contains. The strongest individual machinery in the fleet, and not because of capability: it carries internally what the coding agents leave to their harness.

- A *charter* (`charter.py`) naming the three things that define “better” as never touched — the split, the metric, the judge — because an autonomous agent given a benchmark and a night will find the shortest path to a better number, and the shortest path is almost always to change what “better” means.
- *Validation that can refuse* (`improvement.py`): a candidate becomes an improvement by surviving six checks rather than by winning; the seed rule is load-bearing, since in one field a defended effect is $+0.023$ against a ± 0.027 seed spread; and `NO_CHANGE` is a valid output, so a fortnight of them is a result about the method.
- A *self-model with refusals* (`selfmodel.py`), a structured-ignorance map of claims and which nobody has checked (`fieldmap.py`), a budget that stops rather than asks (`budget.py`), switch-gated autonomy with three ledgers never summed (`dual.py`), and a candidate generator added after a first night scored the incumbent against itself and reported zero (`search.py`).

Outside. The corpus and its held-out split, the reference method, and the owner’s signature on every adoption. Each benchmark reads a measured corpus and refuses to run without it, naming the command that fetches it, because a synthetic substitute produces numbers that look like results and are not. Two of the reference methods fail on real data after passing on synthetic data, and each synthetic pass had been arranged — by the author, writing the generator — to agree with the story the design already told.

Promotes. Stage by stage: the agents are at stage 1 of 5 (proposing; the benchmark verifies, the person signs). Stage 2 requires independent verifier agents; stage 3, agents verifying agents under a signed policy. The ordering is not a roadmap with dates, because *verification is the bottleneck, never generation*: an agent that proposes faster than anything can check it has not advanced its field, it has moved the queue. One promotion has happened end to end: the night loop found that pill-camera’s frame summary took the wrong quantile, the mechanism was tested rather than asserted, and the owner signed the adoption.

The two tests, and where each agent fails them

Each agent is built to be *useful* — a practitioner would run it — and *accepted* — a reviewer for the field would not dismiss it. No agent passes both yet, and each page names the first objection: “you have done one modality and called it a field”; “detectability is your proxy, not a reader”; “held-out molecules are not held-out targets”; “show me calibration”; “an abnormality is not a finding I can put in a report”; “so what?”. Three are blocked on things compute cannot buy — a data agreement, a reader study, more videos — and naming which is which stops the queue being reordered toward whatever is tractable.

6.8 The delegation-level executors DL0–DL3

A different design axis: not one agent climbing, but one binary per level, behind one executor interface, so the tier is real rather than named. `dli-dl0` executes one declared operation with no strategy; `dli-dl1` a short local plan over a known procedure; `dli-dl2` multi-step work with persistent state and ordinary recovery; `dli-dl3` a strategy constructed from an outcome and constraints, with replanning. `dli-accept` runs the acceptance checks, and runs them at DL0.

The property that makes the tier real. A DL0 step must not cost a model call. `dli-dl0` and `dli-accept` run correctly on a machine with no API key and no network, and the claim is checkable offline in an empty network namespace with the keys scrubbed. Each level refuses what is above it *before*

touching a model client, because an executor that calls a model in order to decide to refuse has already attempted the contract. Exit codes are part of the contract: 0 success, 2 usage, 3 specification refusal, 4 self-certification refusal (a record naming its performer as its own verifier), 64 over-level refusal, 70 not implemented.

Outside. The verdict. `exec` reports what was done, never whether it was right; a completed acceptance comparison exits 0 whether or not the bytes matched, and the match is in the evidence.

What has been measured. 88 delegation episodes — four independent implementations across 22 benchmark rows — with 74 accepted by a locus that did not build the systems against criteria compared byte for byte, 53 of 88 recorded `uncertain` rather than `pass` because verdicts were issued only where the oracle ran, and seven κ -cells reported with Wilson intervals at sample sizes of three to eight. Two qualifications were found after first publication and recorded rather than dropped: the acceptance criterion was vacuous on 10 of 22 rows (a directive about where the criterion must live, not a condition), and the three intervention-budget cells differed only by label because zero interventions were raised. No delegation level is certified by that repository, and it says so.

6.9 A cross-cutting rule: the fifth harness

Four executors appear above. The fifth harness is plain Python with a timer, and it is the one most of these agents actually need. The eligibility gate, the fabrication diff, the negative control on outward calls, the solicitation scout, `dli-dl0` and `dli-accept` are all model-free. The routing rule follows: use the cheapest harness that holds, and never let a step a script can do deterministically cost a model call, or the DL0 tier is decorative and the cost argument in the level framework is unearned.

7 The Catalogue

Two tables. The first lists what can be downloaded today, with the command that installs it. The second lists the fleet with target and measured profiles. The columns are deliberately separate: a download link is not a level, and a target is not a measurement.

Artifact	What it is	Where
AI4Science	the framework: chat session in the style of Claude Code, deterministic CLI, the delegation harness (contract, criterion, reversible, acceptor, escalate, router, compress), the ladder runner, the research-agent modules and the sarsi chassis	https://github.com/integritynoble/AI4Science <code>pipx install "pwm-ai4science[claude]"</code> (PyPI 1.0.0); one-line installer in the README. The research-agent and sarsi packages are in the source tree and <i>not yet in the published wheel</i> : from a checkout, <code>pip install ".[research-agents]"</code>
Delegation ladder	HG0–HG3 as built, with the superset test and the curve runner	in AI4Science: <code>python -m ai4science.harness.agents.delegation.run_ladder -seeds 0-1</code>
DL0–DL3 executors	one binary per delegation level plus <code>dli-accept</code> ; DL0 and accept run with no model and no network	https://github.com/integritynoble/intelligence-level <code>dist/dli.pyz</code> (zipapp) or <code>dist/dli_dist-0.4.0-py3-none-any.whl</code> , with SHA256SUMS and a provenance manifest
Field agents (standalone)	eight standalone CLIs, each also embeddable in AI4Science: <code>pwm-unified</code> , <code>pwm-research</code> , <code>pwm-paper</code> , <code>pwm-imaging</code> , <code>pwm-cancer</code> , <code>pwm-drug</code> , <code>pwm-claude-gpu</code> , <code>pwm-codex-gpu</code>	PyPI <code>pwm-agent-unified</code> , <code>-research</code> , <code>-paper</code> , <code>-imaging</code> , <code>-cancer</code> , <code>-drug</code> , <code>-claude-gpu</code> , <code>-codex-gpu</code> (all 0.1.0): <code>pip install pwm-agent-<name></code> then <code>pwm-<name> login</code> ; sources at <a href="https://github.com/integritynoble/pwm-agent-<name>">https://github.com/integritynoble/pwm-agent-<name>
Claude Code, standalone	the official Claude Code as a binary with no Node.js requirement, for use behind the harness	https://github.com/integritynoble/claude-pwm (releases; latest 2.1.220)
Personal Singularity	the console that hosts the fleet across machines: <code>doctor</code> , <code>up</code> , <code>status</code>	in AI4Science: <code>curl -fsSL https://raw.githubusercontent.com/integritynoble/AI4Science/main/install-singularity.sh sh</code>
Codex, exchange-routed	the Codex CLI fork that accepts a base URL and an exchange key, so the same motor runs through a different provider	https://github.com/integritynoble/codex-pwm (release builds)
OpenClaw, exchange-routed	the openclaw agent runtime routed through the exchange	https://github.com/integritynoble/openclaw-pwm <code>npm install -g openclaw openclaw-pwm</code>
Framework and this paper	the measurement framework v2.3, its datasets and audit tools, and the corpus this paper belongs to	https://github.com/integritynoble/sarsi-intelligence-level

Table 6: What can be downloaded, and how. Versions are those current at the date of this draft.

Agent	Unit	Motor	Target [<i>C, I/M, O, T·H, SA</i>]	Measured today	Runnable
sarsi-worker	indiv.	brain; dispatches to any motor	C3, I2/M3, O2, T3·H1, SA3	M1 cand.; O0; SA1 prov.; rest unmeasured	yes, from a source check-out
sarsi-claude	indiv.	Claude Code (ACP)	C2–3, I0/M0, O0, T2·H1, SA1	O0; SA0; rest unmeasured	yes
sarsi-ai4sci	indiv.	AI4Science	C2–3, I0/M0, O0, T2·H1, SA1	declared on 9 accounts, binds nothing	declared
sarsi-open	indiv.	OpenCode (ACP)	C2, I0/M0, O0, T1–2·H1, SA1	binary present on one host; no charter; never run	not yet
funding	org.	Python gate + Claude Code draft	C2, I2/M3, O1, T2·H2, SA2	identity real; O0; SA0; binds no runtime	design
job	org.	as funding + fabrication diff	C2, I2/M3, O1, T2·H2, SA2	no source CV; unfilled identity	design
social-media	org.	drafting only, no credential	C2, I1/M1, O1, T1·H2 capped , SA2	unfilled identity; correct by absence	design
learning	org.	—	C2, I2/M3, O1, T2·H2, SA2	unfilled identity	design
8 research agents	org.	AI4Science modules; Claude Code for search	C4, I2/M3, O3, T3·H1, SA3 (C5/I5/T5 spec-only)	machinery built, stage 1 of 5; field benchmarks: 5 pass, 2 fail, 1 design-only; fleet deployment design-only; O0	machinery yes (source); deployment design
dli-dl0...dl3	indiv.	none (DL0), model client (DL1–3)	DL0–DL3 by construction	88 episodes, 74 accepted, two qualifications; no level certified	yes (binary)

Table 7: The fleet with target and measured profiles. Every U cell is empty and every $T·H·p$ cell is unmeasured; “M1 cand.” is configuration evidence, not a level. The fleet’s charters and per-agent designs live in a private repository; the mechanisms they rely on are the public ones in Table 6.

8 How to Measure What You Built

Every design above ends in a test that promotes it, and none of those tests has been run on the fleet. This section states the order that costs least, and the protocol it follows is released with the framework’s corpus.

1. **Freeze.** Model identity from a live call, the agent’s runtime block, charter hashes, tool grant, a memory snapshot restored before every episode, host and gateway, and an explicit timeout. The framework’s regime-switch comparison has an unnamed frontier executor because this step was skipped once.
2. **Seven apparatus checks before any episode**, each a positive control: the timeout kills a grandchild’s process group; termination reasons are recorded; the gate opens on a known pass and closes on a known failure; both ends agree on versions; restore is clean; no session survives its episode.
3. **The restart probe on sarsi-worker first.** It is the cheapest test in the fleet, the only one whose instrument is already written, and it settles the coordinate the fleet table once misreported. Plant a fact, kill the process group, restart, ask for the fact and where it came from, against a floor agent with an empty store.
4. **The delegation sweep at H1**, six task classes at T0–T3, four seeds, scored with the delivered-outcome primitive under predeclared weights, with per-band headroom and a concordance audit on any item two

executors fail identically. Three motors are the minimum roster for that audit; this fleet supplies exactly three.

5. **The coordinate probes** for O (registry and ablation; N/A for individual units, recorded as N/A), SA (limits before attempt, mechanism of failure, forecast before run) and C (paired familiar and unfamiliar items).
6. **Report** per pair: U^* , HLIS over the declared dimensions, the profile, the named bottleneck, reliability with an interval, the frontier at H0/H1/H2, the false-completion rate, the resource envelope, headroom, and concordance-audit status.

The expected outcome from configuration evidence is $U1$ for `sarsi-worker` if the restart probe passes, and $U0$ for everything else. That is not a disappointing result. It is the first non-empty cell, earned by a locus that did not do the work, on tasks it did not author — and it is the number every design in this paper is written to make possible.

9 Limitations

- **Nothing here is a certified level.** Every target is a target; every “built” is built, not measured. The fleet table is honest about that and so is this sentence.
- **Two of the four motors have never run a step here.** An OpenCode binary is present on one host and has never been exercised, and Hermes exists only as an unregistered adapter with nothing installed behind it. The motor-independence claim is therefore demonstrated across two commercial motors and asserted for the open ones.
- **The arms measurement is one family.** Fifteen instances, one intervention budget, one host. It establishes that the harness moved the outcome; it does not estimate a rate.
- **The DL binaries’ headline was measured before a fix.** The 74/88 figure predates the repair of a vacuous criterion on 10 rows and has not been re-measured.
- **Private designs, and designs only.** The per-agent charters and the funding, job, learning and outward-posting designs are described here; their workspaces are not released, and the fleet’s own handoff record states that nothing in those designs has been executed. A reader can rebuild them from §6 and the public mechanisms, and cannot download them.
- **The published wheel is stale.** The research-agent and `sarsi` packages are in the public source tree and not in the PyPI distribution current at this draft.
- **Discovery levels have no runnable items.** $C5$, $I5$ and $T5$ targets are specification-only until a sealed-mechanism family is bound to the research agents.

10 Conclusion

The intelligence level of an agent is a property of the pair, and the executor is the part of the pair that is easiest to swap. Behind one protocol, Claude Code, Codex, OpenCode and Hermes differ in proposal quality and in nothing that moves a level. What moves a level is built around them: a criterion registered before the work and sealed when it starts, an acceptor in another process on a copy, a snapshot before the first mutation, a router that learns only from independent verdicts, a memory a restart can test, a roster an ablation can test, a self-model that refuses, and the placement rule that puts verification in the hands of executors with nothing to write to themselves. Those mechanisms are released, one ladder built from them has been measured, and a fleet of agents — a brain, three motors, three organizational agents, eight field agents and four level executors — is catalogued with what each targets, what each has shown, and where each can be obtained.

What the fleet has not yet done is earn a level, and the paper does not pretend otherwise. The first one is one restart away.

Availability

All public artifacts are listed in Table 6 with their repositories and package names. The measurement framework, its datasets and audit tools, and the measurement protocol for the first three agents are in the `sarsi-intelligence-level` corpus. The author’s fleet configuration and per-agent charters are in a private repository; requests for access should be addressed to the correspondence address.

Conflicts and funding

The author reports no funding specific to this work and no competing interest. All agents described were built and run on the author’s own infrastructure.

References

- [1] Chengshuai Yang. Capability is not a level: A structural taxonomy of individual and organizational intelligence, with placements for current AI agents. Draft, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [2] Chengshuai Yang. Designing LLM-driven agents across individual and organizational intelligence levels. Working paper and build specification, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.
- [3] Chengshuai Yang and Ting Xue. Measuring the unified intelligence level: A cumulative, harness-measurable hierarchy. *Version 2.3*, <https://github.com/integritynoble/sarsi-intelligence-level>, 2026.